


Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU

# Assignment 4: Memory hierarchy, square roots

Programming Techniques for Supercomputers (PTfS)

*Muhammad Omar*
Student IdM: to37gulo
Student HPC-Account: ptfs356v

*Gehan Younes*
Student IdM: qi39dyry
Student HPC-Account: ptfs437v

May 27, 2026

# Q1: Optimizing for Code Balance (40 crd)

## (a) Rewrite and change in memory code balance (15 crd)

**Optimization: loop fusion (2-way unrolled).** Loop 1 writes `a[i]`; loop 2 immediately re-reads it. Fusing them keeps `a[i]` in a register, removing the round-trip of `a` through memory:

```
for(i=0; i<N; i+=2) {
  a[i]   += b[i]   * c[i];
  a[i+1] += b[i+1] * c[i+1];
  z[i]    = a[i]   * 0.5f;     // a[i] still in register
}
```

**Memory code balance.** Single precision (4 B/elem), arrays $\gg$ LLC, WA enabled. Total flops: $2N + N/2 = \frac{5}{2}N$.

*Before (two separate loops):*

- `a`: load+store in loop 1, plus full reload in loop 2 $= 12N$ B
- `b`, `c`: each $4N$ B
- `z`: WA-load + store $= 8N$ B
- Total: $V = 28N$ B

$$\boxed{B_c^{\text{before}} = \frac{28N}{(5/2)N} = 11.2 \text{ B/F}}$$

*After fusion:* the loop-2 reload of `a` is eliminated ($-4N$ B).

$$V = 24N \text{ B} \quad \Rightarrow \quad \boxed{B_c^{\text{after}} = \frac{24N}{(5/2)N} = 9.6 \text{ B/F}}$$

Reduction: $\Delta B_c = 1.6$ B/F ($\approx 14.3\%$).

## (b) Cycles per 16 iterations on the hypothetical CPU (25 crd)

**Work unit:** 16 iterations = exactly one 64 B cache line each of `a, b, c, z`. With inclusive caches and arrays $\gg$ LLC, the same data volume must traverse every level.

| Array | Operation | Bytes per 16 iter |
|---|---|---|
| `a` | RMW: 1 CL in + 1 CL out | 128 |
| `b` | 1 CL load | 64 |
| `c` | 1 CL load | 64 |
| `z` | WA-load + evict | 128 |
| **Total** | | **384 B** |

Memory bandwidth in cy units: 110 GB/s/2.4 GHz $\approx$ 45.83 B/cy.

$$\boxed{T_{L1\text{–}L2} = \frac{384}{64} = 6 \text{ cy}, \quad T_{L2\text{–}L3} = \frac{384}{16} = 24 \text{ cy}, \quad T_{L3\text{–Mem}} = \frac{384}{45.83} \approx 8.4 \text{ cy}.}$$

**Bottleneck:** L2↔L3 (24 cy, $\approx 63\%$ of the no-overlap total). Surprisingly, main memory is *not* the limiter on this CPU model.

## Q2: In-Memory Code Balance (35 crd)

Assumptions: arrays ≫ any cache, WA on stores, no NT stores, scalars held in registers.

| | Loop body | | Per-iter traffic | $B_c$ |
|---|---|---|---|---|
| (a) | `s = s + a[i];` | SP | 4 B / 1 F | **4 B/F** |
| (b) | `s = s + a[i]*a[i];` | DP | 8 B / 2 F | **4 B/F** |
| (c) | `a[i][j] += s*c[i][j];` | SP, inner $j$ | (4+4+4) B / 2 F | **6 B/F** |
| (d) | `a[i] = s*b[i];` | int32 | (4+4+4) B / 1 op | **12 B/op** |

Notes: (b) `a[i]` is loaded once and reused for the square. (c) WA is folded into the load of `a`. (d) Store-only ⇒ *explicit* WA load + store. Integer code has no flops; we report B/op.

**(e) `x[i] = s*v[index[i]];` DP, DP, int32 – 15 crd** Definite traffic per iter (independent of indices): load `index` (4 B) + store `x` (8 B) + WA `x` (8 B) = 20 B, with 1 Flop. The `v`-traffic depends entirely on the access pattern:

- **Best** (sequential, e.g. `index[i]==i`): full CL reuse ⇒ 8 B/iter for `v`.
- **Worst** (random over a large range): every access loads a full 64-B CL, of which only 8 B are used ⇒ 64 B/iter for `v`.

$$B_c^{\text{best}} = \frac{20+8}{1} = 28 \text{ B/F} \quad \leq \quad B_c \quad \leq \quad B_c^{\text{worst}} = \frac{20+64}{1} = 84 \text{ B/F}$$

## Q3: SQRT Latency on Fritz (25 crd)

**Hardware.** Fritz: 2× Intel Xeon Platinum 8360Y (Ice Lake-SP), 36 cores @ 2.4 GHz, AVX-512 with two 512-bit FMA units per core [1].

**Build & run.** Compiled with `icc` 2021.10.0 (module `intel/2023.2.1`); each binary pinned to core 0 with `likwid-pin`:

```
icc -O3 -xCORE-AVX512 -qopt-zmm-usage=high integrate.c -o pi_avx512 -lm
icc -O3 -xAVX2                             integrate.c -o pi_avx2   -lm
icc -O3 -xSSE4.2                           integrate.c -o pi_sse    -lm
icc -O1 -no-vec                            integrate.c -o pi_scalar -lm
```

### (10 crd) Measured cycles per iteration

Stable values at $N \geq 4 \times 10^7$ (job 3624598, node f1018):

| Variant | Width | Measured cy/it | Per-instruction |
|---|---|---|---|
| AVX-512 | 8 DP | **1.28** | 10.2 cy/inst |
| AVX2 | 4 DP | **3.01** | 12.0 cy/inst |
| SSE 4.2 | 2 DP | **2.06** | 4.1 cy/inst |
| Scalar | 1 DP | **6.19** | 6.2 cy/inst |

### (15 crd) Is this what we expected?

**No – two clear surprises:** (i) AVX-512 (1.28) is fastest, even beating naive SIMD scaling; (ii) AVX2 (3.01) is *slower* than the narrower SSE (2.06).

**Naive expectation.** If each loop iteration executed a literal `(V)SQRTPD`, then cy/it = inst-throughput/$W$. Using published Ice Lake throughputs [2] (SQRTSD/XMM $\sim$ 6, YMM 9, ZMM $\sim$ 18):

$$\text{predicted: } 6.0 \text{ (scalar)} \rightarrow 3.0 \text{ (SSE)} \rightarrow 2.25 \text{ (AVX2)} \rightarrow 2.25 \text{ (AVX-512)}.$$

Only the scalar prediction matches. So the compiler must *not* be emitting plain `VSQRTPD` for the vectorized binaries.

**What is actually happening.**

- **Scalar (6.19):** `-O1 -no-vec` keeps a literal `SQRTSD` on the non-pipelined sqrt unit (port 0) – matches the documented $\sim$6 cy. This is the only case where the lecture's "sqrt dominates" assumption is literally exercised.
- **SSE (2.06) and AVX-512 (1.28):** at `-O3` the Intel compiler routes vectorized `sqrt()` through the SVML library, which replaces `vsqrtpd` with a reciprocal-sqrt estimate plus a short Newton-Raphson refinement on the fully-pipelined FMA units. Per-element cost therefore drops well below the slow iterative `vsqrtpd`, and scales (sub-)linearly with SIMD width (2→8 elements gives a 1.6× speedup, limited by the dependent FMA chain in the Newton step).
- **AVX2 anomaly (3.01 > SSE 2.06):** AVX2 has no double-precision reciprocal-sqrt estimate (`vrsqrt14pd` is AVX-512 only), so SVML's AVX2 path falls back to (essentially) plain `VSQRTPD YMM`. Combined with mild AVX2-license frequency throttling on Ice Lake-SP [3], the effective per-element cost inflates from the naive 2.25 to the measured 3.01. Meanwhile SSE runs at the unrestricted base clock with the faster SVML SSE sequence, so 128-bit beats 256-bit here.

A nice corroborating signal: in the SSE output, `cy/it` drops from 3.01 to 2.06 at $N \approx 1.7 \times 10^8$ – the chip visibly exits the AVX-license throttled state once a long, purely 128-bit run convinces the power-management unit no wide SIMD code is coming.

**Take-away.** The naive "cy/it = `vsqrtpd`-throughput /$W$" model is too simplistic for vectorized binaries: it ignores (1) SVML's reciprocal-sqrt approximation on fully pipelined FMA units, and (2) license-based frequency throttling on AVX2/AVX-512.

## References

[1] NHR@FAU HPC Documentation: *Fritz cluster*. `https://doc.nhr.fau.de/clusters/fritz/`

[2] A. Abel and J. Reineke. *uops.info*. `https://uops.info/`

[3] D. Schor. *Intel Launches 3rd Gen Ice Lake Xeon Scalable*, WikiChip Fuse, 2021. `https://fuse.wikichip.org/news/4734/intel-launches-3rd-gen-ice-lake-xeon-scalable/`